



LEADERSHIP
COMPUTING
FACILITY

August 24, 2026

HPC-QC Integration Avenues for Quantum PDE Solvers in Fluid Dynamics

OpenQSE 2026

Murali Gopalakrishnan Meena

National Center for Computational Sciences, ORNL



U.S. DEPARTMENT
of **ENERGY**

ORNL IS MANAGED BY UT-BATTELLE LLC
FOR THE US DEPARTMENT OF ENERGY

FRONTIER



*This research used resources of the **Oak Ridge Leadership Computing Facility** at the Oak Ridge National Laboratory, which is supported by the Advanced Scientific Computing Research programs in the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.*

ORNL QCFD team and collaborators



Kalyan
Gottiparthi



Chao
Lu



Toño
Coello Pérez



Amir
Shehata



Seongmin
Kim



Pooja
Rao
NVIDIA



Matthew
Horner
Aegiq



Callum
Duncan



Mohammad
Alhawwary
GE Aerospace



Andy
Gallo



Michael
Sandoval



In-Saeng
Suh



Antigoni
Georgiadou



Matt
Norman



Tom
Beck



Xinfeng
Gao
UVA



Yu
Zhang
LANL



Manu
Chaudhary
Illinois State
Univ.



Esam
El-Araby
Univ. Kansas

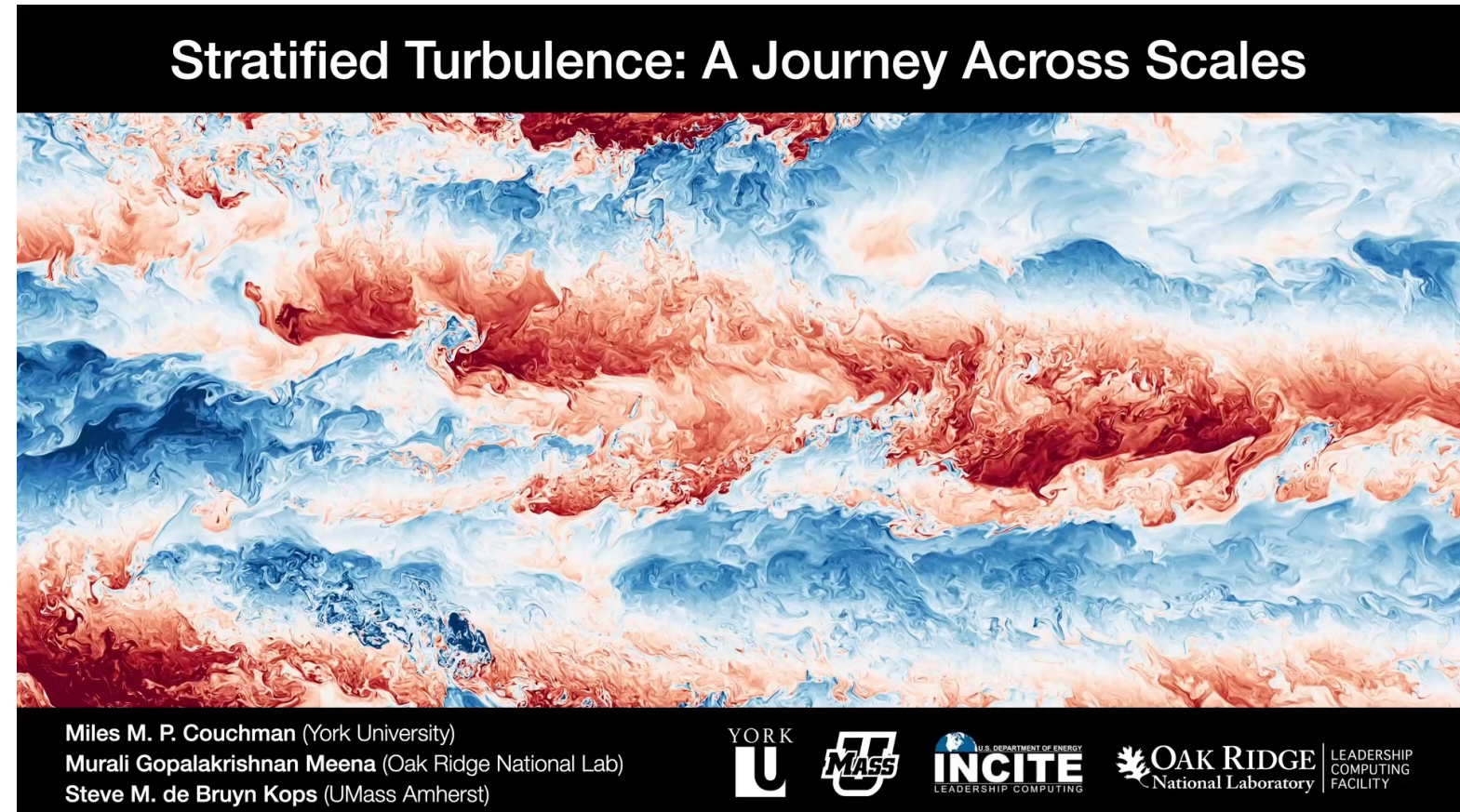
ORNL - OLCF

Introduction & Motivation

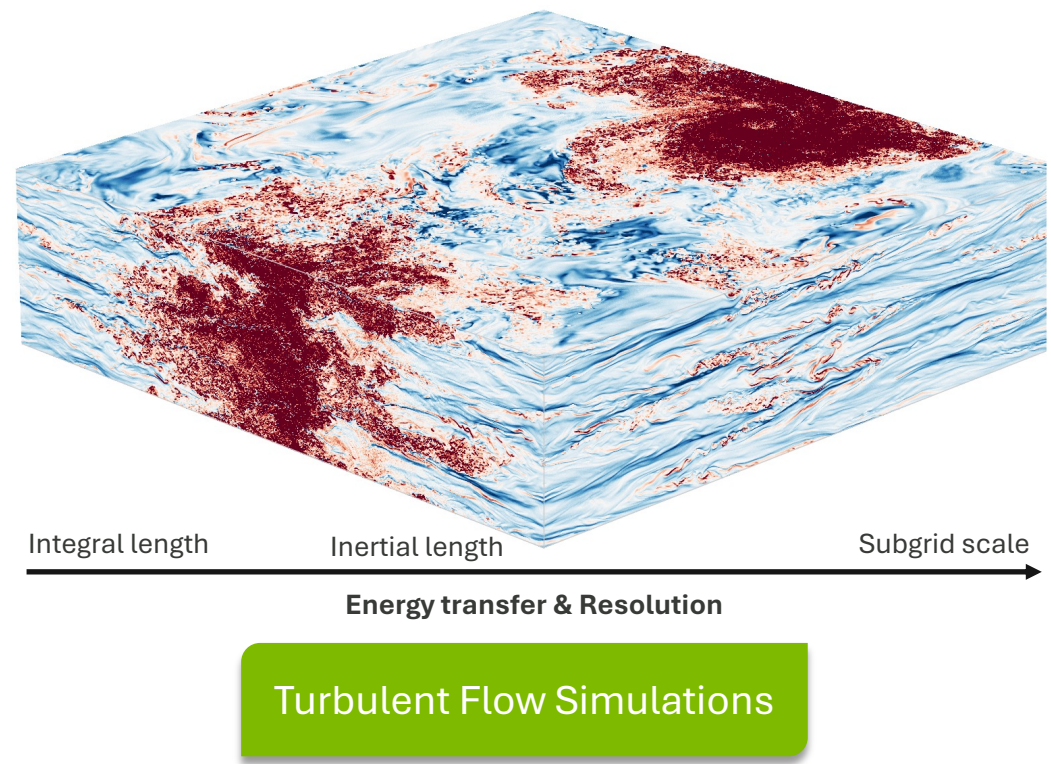
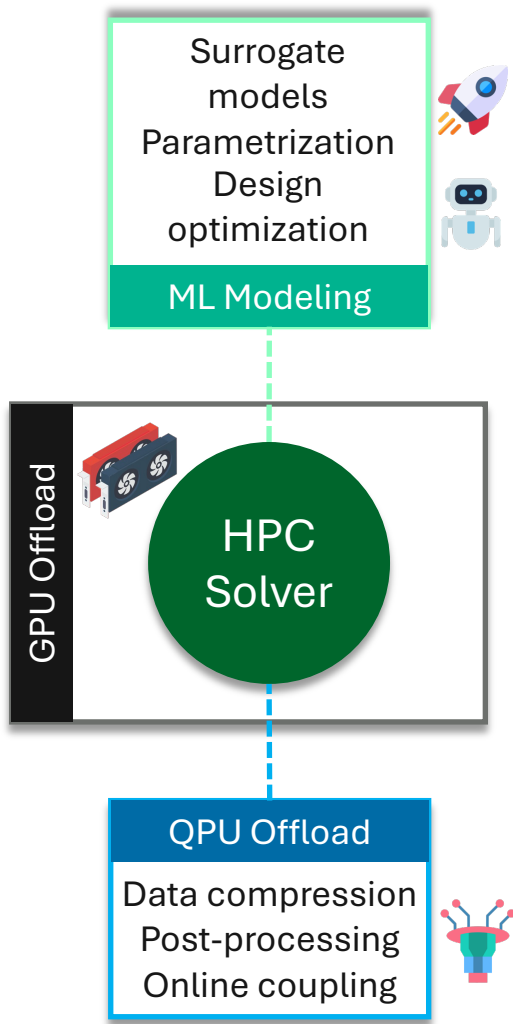
Fluid dynamics & Extreme-scale computation

Grid resolution can get prohibitively expensive to simulate for practical fluid flow problems

- 3D stratified turbulence model for oceanographic flow^[1]
- $37k \times 37k \times 4k \approx 6.6 \times 10^{12} > 2^{42}$ grid points
- 100 TB per snapshot
- Turbulence related projects: 35-45% of 2023 OLCF Frontier allocation



Accelerating fluid flow simulations: An integrated ecosystem of HPC+AI+QC



Governing equations of fluid flow comprise of linear & nonlinear terms that can be solved numerically

Conservation of mass

$$\nabla \cdot \mathbf{u} = 0,$$

Conservation of momentum

$$\frac{\partial \mathbf{u}}{\partial t} = - \underbrace{\mathbf{u} \cdot \nabla \mathbf{u}}_{\text{nonlinear term}} - \underbrace{\nabla p + \frac{1}{Re} \nabla^2 \mathbf{u}}_{\text{linear terms}} + \mathbf{s}$$

Pressure Poisson equation

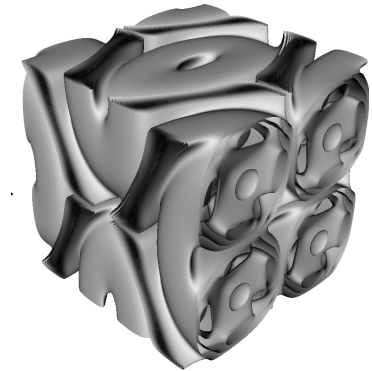
$$\nabla^2 p = - \underbrace{\nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u})}_{\text{nonlinear term}}$$

The Navier—Stokes Equations

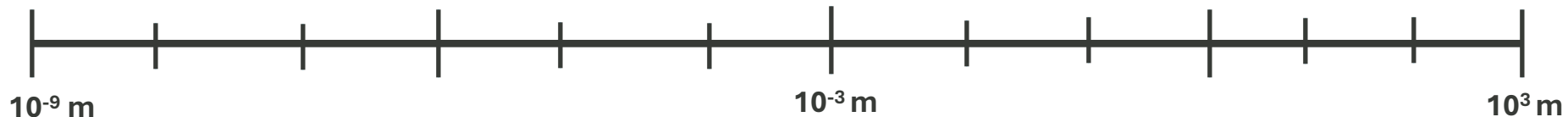


Smallest scale for reactions and viscous dissipation

Subgrid scales Resolved scales



Large scale structures



Governing equations are discretized to create a set of algebraic equations & assembled into $Ax = b$

$$\nabla \cdot \mathbf{u} = 0,$$

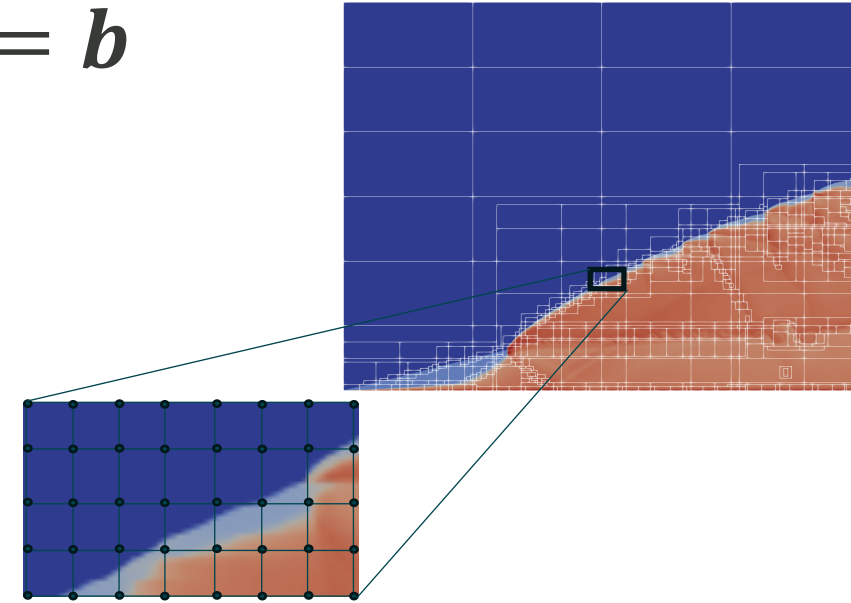
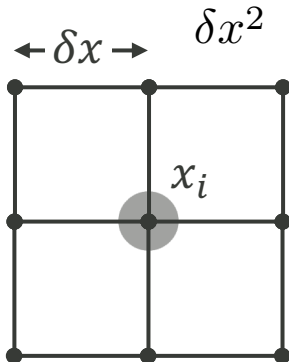
$$\frac{\partial \mathbf{u}}{\partial t} = - \underbrace{\mathbf{u} \cdot \nabla \mathbf{u}}_{\text{nonlinear term}} - \underbrace{\nabla p + \frac{1}{Re} \nabla^2 \mathbf{u}}_{\text{linear terms}} + \mathbf{s}$$

$$\nabla^2 p = - \underbrace{\nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u})}_{\text{nonlinear term}}$$

Finite difference approximation

$$\frac{\partial u_\alpha(x_i)}{\partial x} \simeq \frac{u_\alpha(x_i + \delta x) - u_\alpha(x_i - \delta x)}{2\delta x}$$

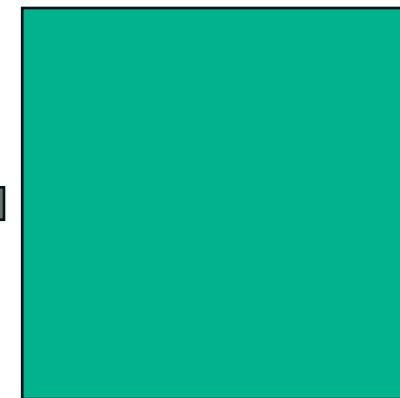
$$\frac{\partial^2 u_\alpha(x_i)}{\partial x^2} \simeq \frac{u_\alpha(x_i + \delta x) + u_\alpha(x_i - \delta x) - 2u_\alpha(x_i)}{\delta x^2}$$



Desai et al., Combust. Flame, vol 257, 2023



Linear Solver



A

$\times$

$=$



x



b

Quantum linear & nonlinear PDE solvers have the potential to exponentially reduce cost of solving large problems

- Quantum computing applications to fluid flow problems:
 - Lattice simulations: fluid motion modeled as the motion of discrete particles
 - **Continuum simulations**: fluid motion modeled as a continuous field

Linear flow problems

- Linear (ideal) & Linearized PDEs^[1-4]: N—S equations with assumptions
- Use **Quantum Linear System Algorithms (QLSA)**
- Classical: $O(N)$ (or higher for denser non-symmetric matrices)
- Quantum: $O(\log(N))$
- Disadvantage: Enlarged solution space

Nonlinear flow problems

- Tackle nonlinearity of PDEs^[5-10]
- Not generalized
- Limited work

[1] Yepez, *PRE*, 2001

[2] Xu, Daley, Givi, Somma, *AIAA*, 2018

[3] Bharadwaj & Sreenivasan, *PNAS*, 2023

[4] Gopalakrishnan Meena et al., *PoF*, 2024

[5] Leyton & Osborne, *arXiv*, 2008

[6] Gaitan, *NPJ*, 2020

[7] Steijl, *Quantum Comp. & Comm.*, 2020

[8] Oz, et al., *Sci. Rep.*, 2023

[9] Lubasch, et al., *PRA*, 2020

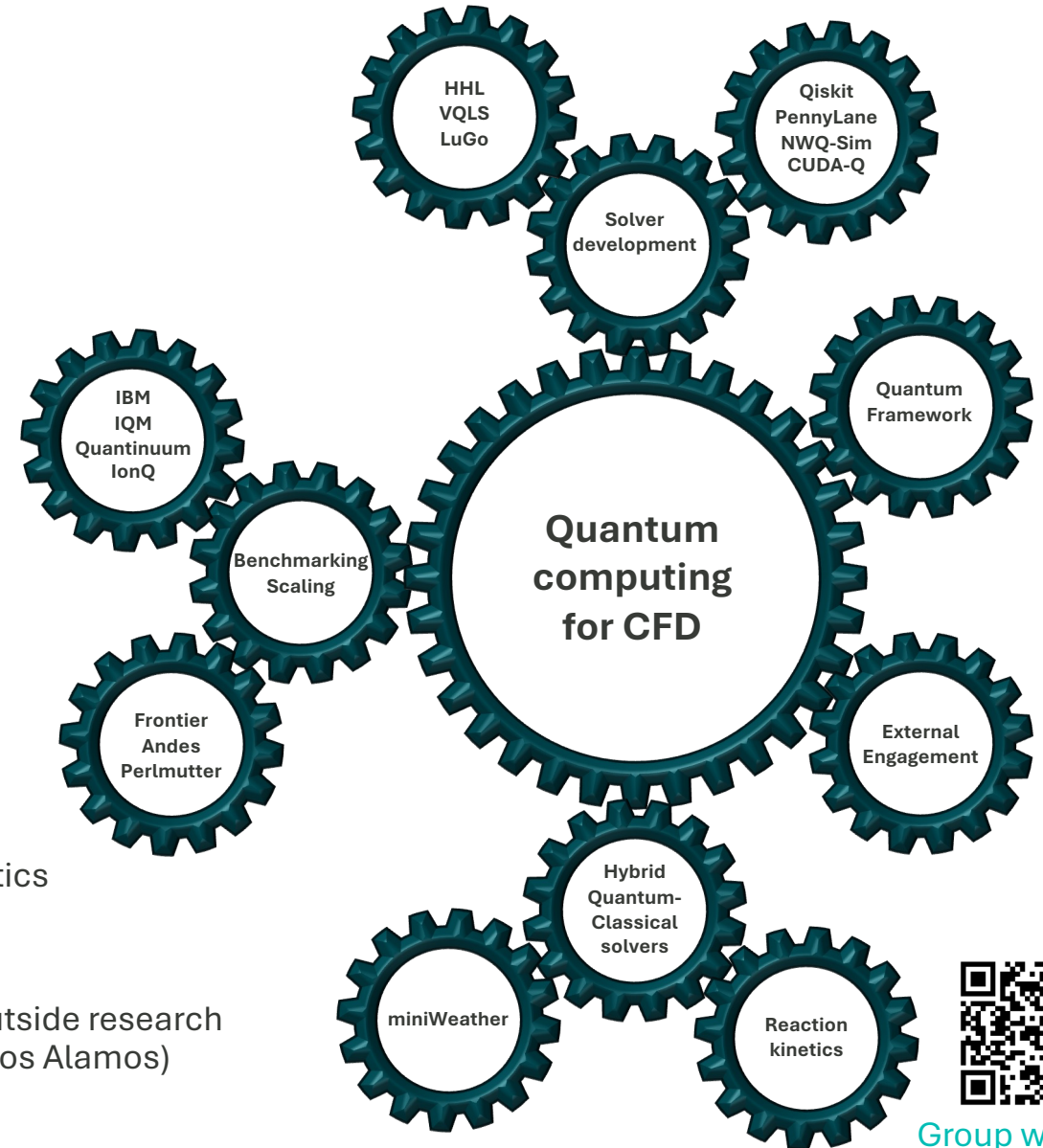
[10] Gopalakrishnan Meena et al., *IEEE QCE*, 2024

Objectives of this presentation

- Disclaimer: We are **not** trying to demonstrate quantum advantage for fluid flow problems
- Current talk objectives:
 - Efforts at OLCF to assess the **applicability of QC for CFD**
 - Overview of various algorithms: **focus on HPC-QC integration patterns**
 - Efforts driven by **hardware** availability & **application** needs
 - NISQ & FTQC
 - Linear PDEs & preliminary efforts on nonlinear PDEs
 - **Collaborative effort** & welcome collaboration

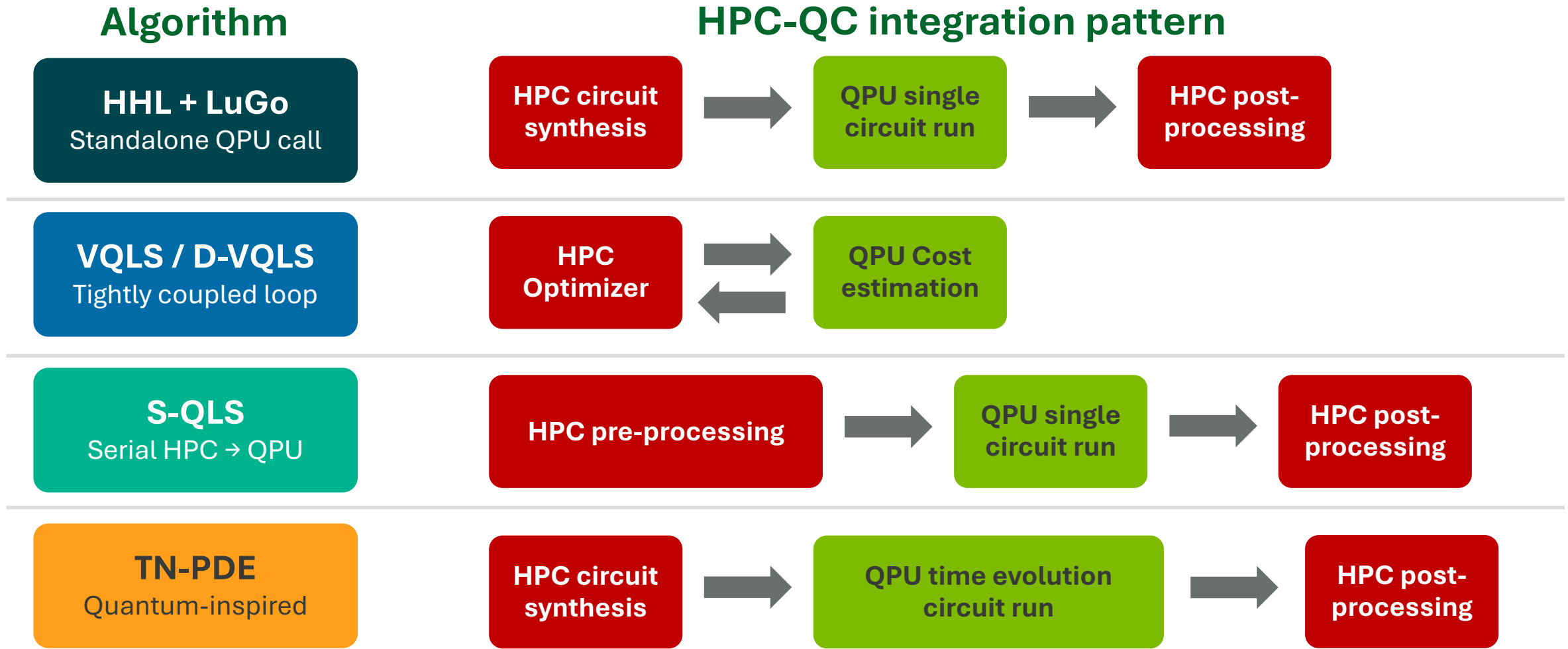
Project Overview:

- **Solver Development**
 - Code development using HHL, VQLS, and other QLSA
 - Algorithm improvement: QPE, TN-based nonlinear solver
- **Benchmarking and scaling**
 - Running on various backends: simulated and real
 - Application development and comparison using different SDKs
- **Quantum framework**
 - Developing and testing with QFw
 - Redesign for efficient resource management
- **Hybrid Quantum-Classical solvers**
 - Coupling classical codes with quantum solver
 - Applications: turbulence closures, tracer tracking, and reaction kinetics
- **External Engagement**
 - Collaboration to develop different applications – within ORNL and outside research groups (GE Aerospace, Aegiq, NVIDIA, UVA, KU, UTC, U Melbourne, Los Alamos)
 - Student competitions, tutorials, and internships



[Group website](#)

Each solver we develop exercises a different mode of HPC-QC integration



Fault-tolerant Quantum Linear Solver Algorithm

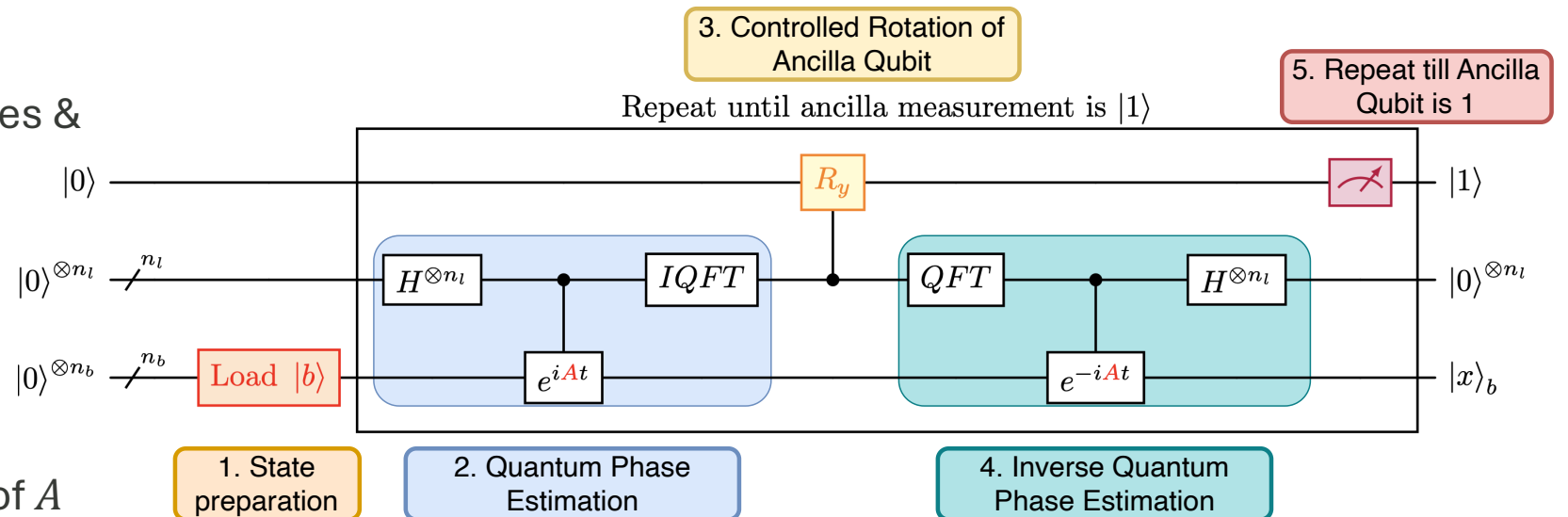
Solving the Hele—Shaw flow using the HHL algorithm

Solving canonical flow problems using the HHL algorithm on superconducting & trapped-ion devices

- A canonical Quantum Linear Systems Algorithm: the [Harrow–Hassidim–Lloyd \(HHL\) algorithm](#)

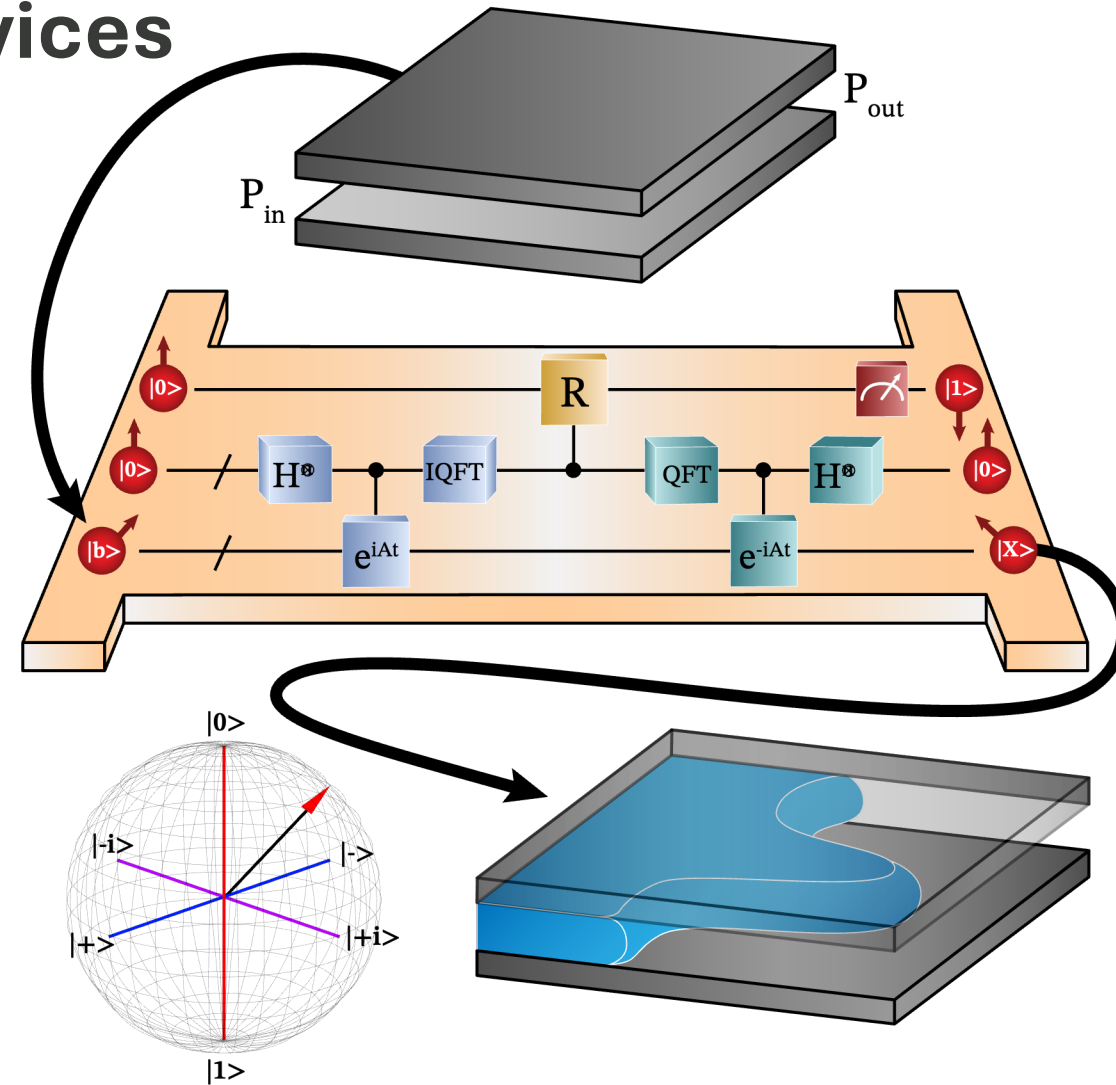
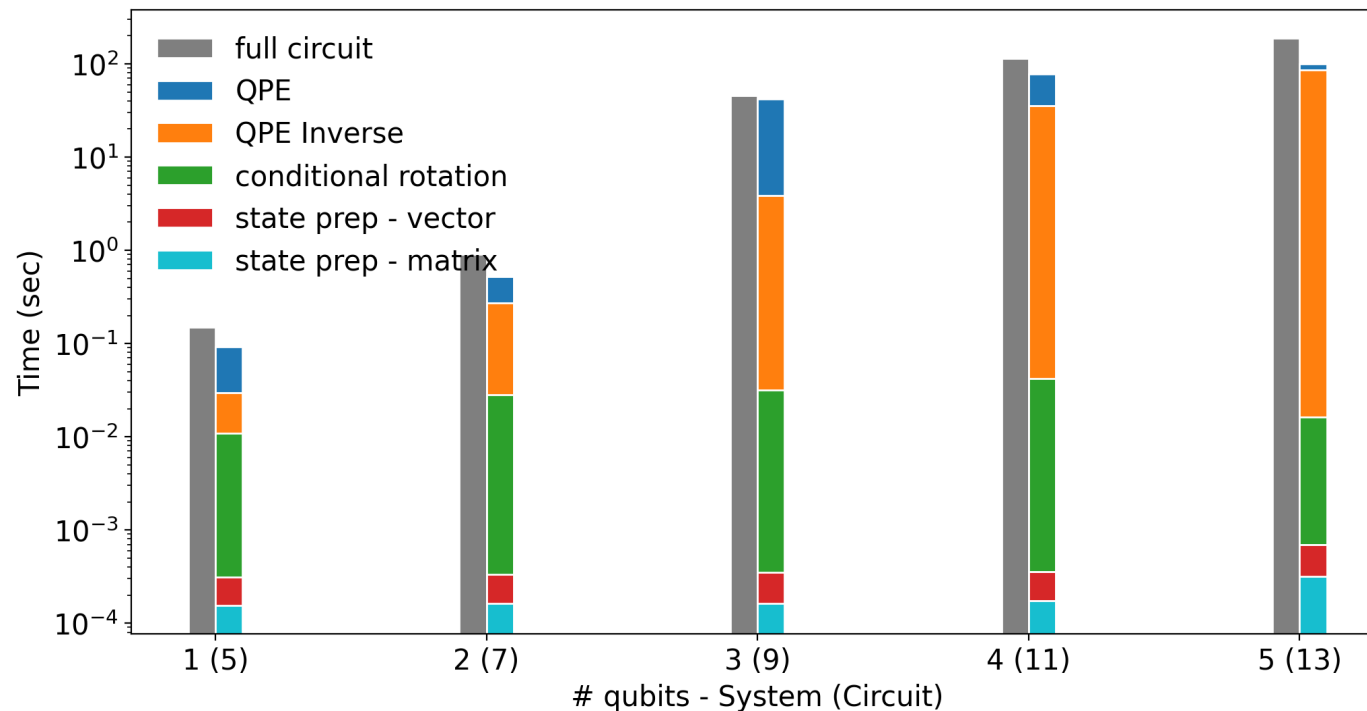
- $\mathbf{x} = A^{-1}\mathbf{b} = \sum_j \lambda_j^{-1} b_j u_j$,
 - λ_j & u_j are the eigenvalues & eigenvectors of A

- Computational complexity:
 - $O(\log(N) s^2 \kappa^2 / \epsilon)$
 - N – size of A
 - s – sparsity of A
 - κ – condition number of A
 - ϵ – accuracy of approximation
 - Up to $\kappa \log(\kappa/\epsilon)$



Solving canonical flow problems using the HHL algorithm on superconducting & trapped-ion devices

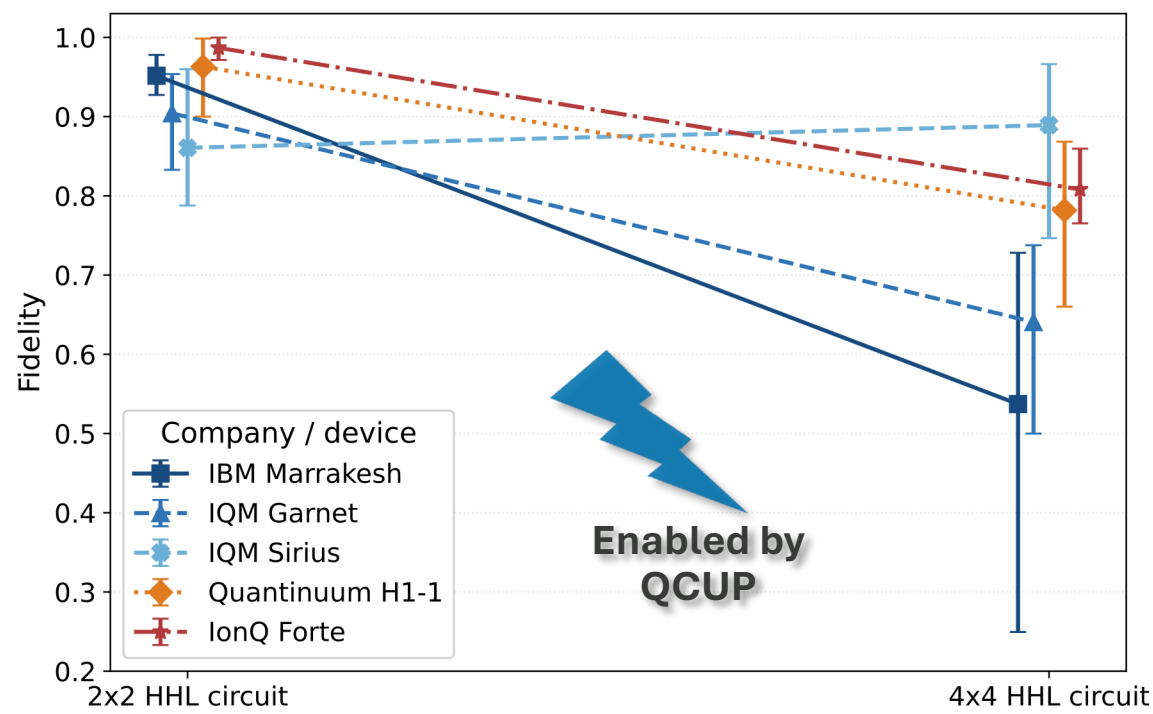
- HHL used to solve canonical fluid dynamics problem (Hele—Shaw flow)
- Challenge:** Accuracy; Cost to generate circuit



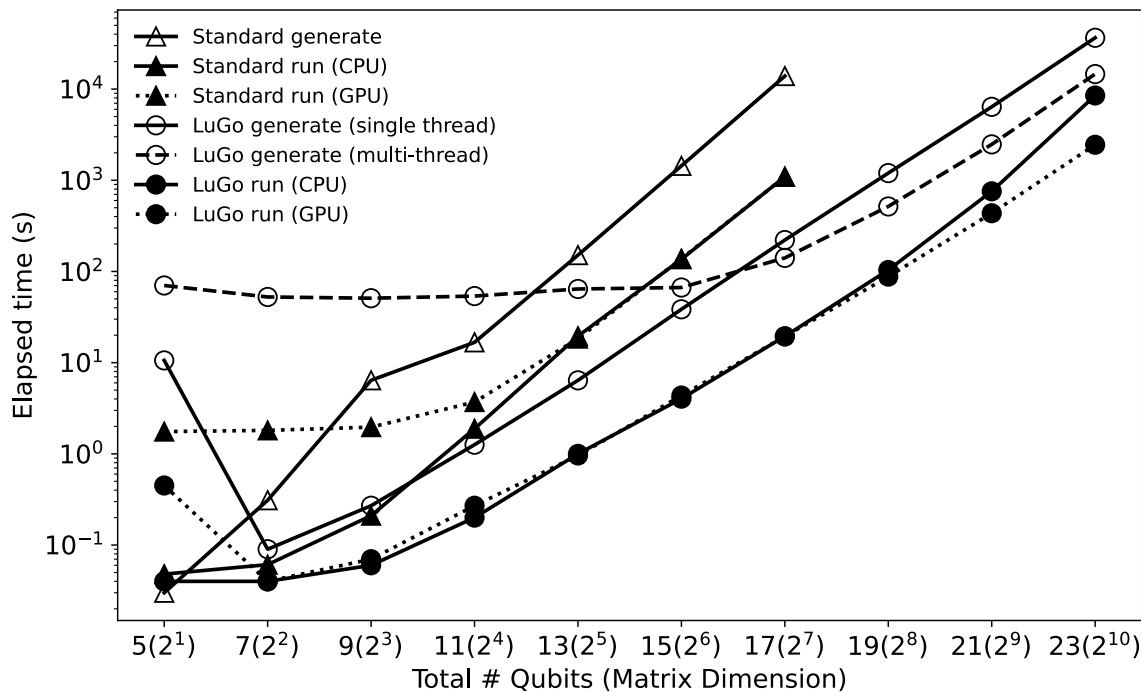
[Gopalakrishnan Meena et al., PoF, 2024](#)

Solving canonical flow problems using the HHL algorithm on superconducting & trapped-ion devices

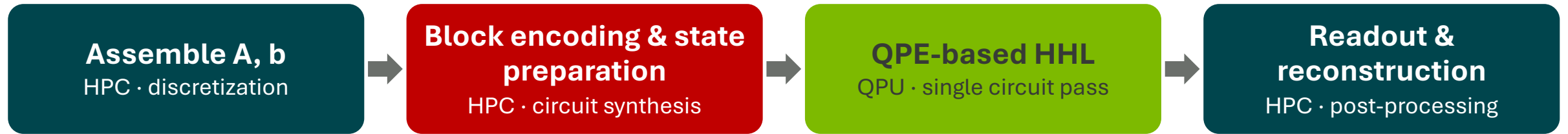
- **LuGo**: an enhanced QPE implementation – enables scalable HHL



Matrix Size	Qubits	Reduction		
		Time	Gate	Depth
$2^1 \times 2^1$	5	$0.51\times$	$1.57\times$	$1.80\times$
$2^2 \times 2^2$	7	$1.48\times$	$2.48\times$	$2.88\times$
$2^3 \times 2^3$	9	$4.86\times$	$5.15\times$	$5.43\times$
$2^4 \times 2^4$	11	$12.95\times$	$8.80\times$	$8.92\times$
$2^5 \times 2^5$	13	$29.13\times$	$17.88\times$	$17.95\times$
$2^6 \times 2^6$	15	$50.68\times$	$31.59\times$	$31.62\times$



Pattern 1: A standalone QPU call — HPC prepares, the QPU solves once



Characteristics of this mode

- **One-directional, serial:** the QPU never waits on a classical decision mid-solve
- **Batch-queue tolerant:** quantum call can be *submit-and-forget*
- **Classical cost:** circuit generation and transpilation
 - LuGo removes redundant QPE repetitions
- **Accuracy**, not scheduling, is the limiting factor at current scales

What the stack must provide

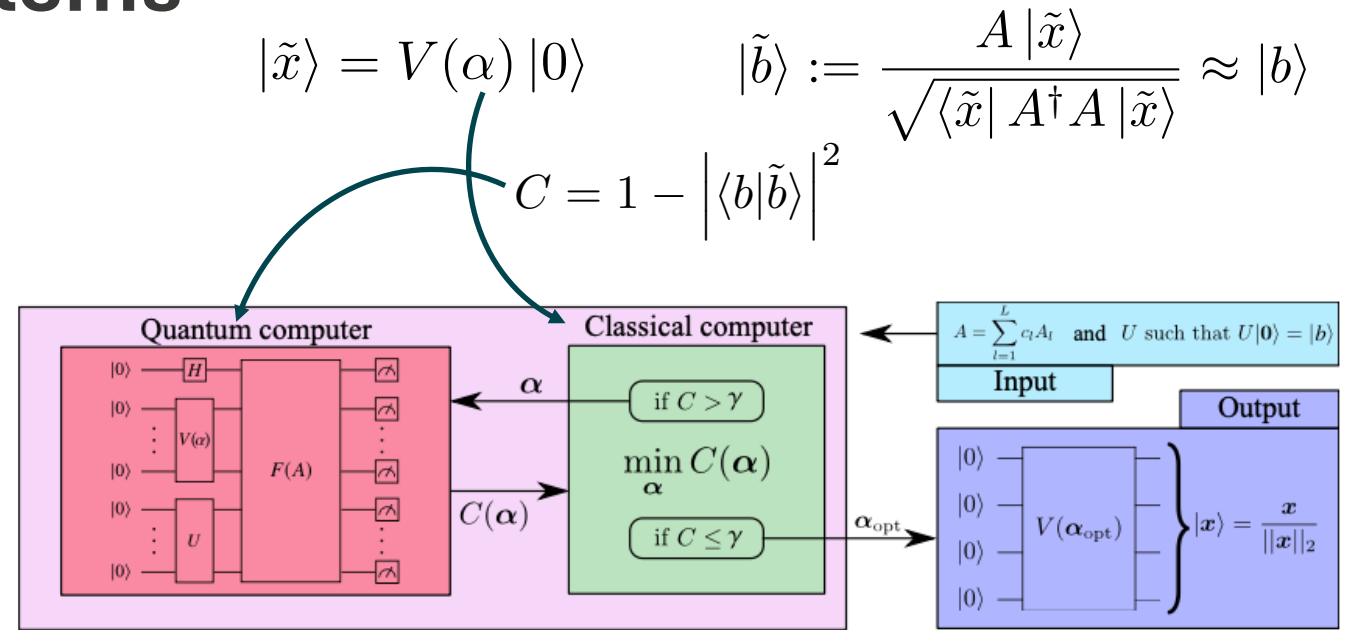
- GPU-accelerated circuit synthesis & transpilation
- Reliable execution of one long circuit, not many short ones
- ≈ 20 – 25 logical qubits and 10^5 – 10^6 gate operations for a 2^{10} system
- Caveat carried to the end of this talk: “standalone” holds only while state preparation fits on a single node

Variational Quantum Algorithm for Linear Solvers

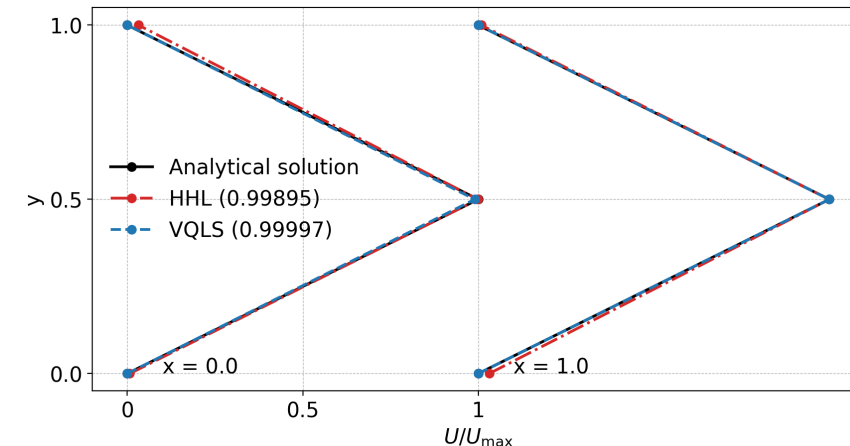
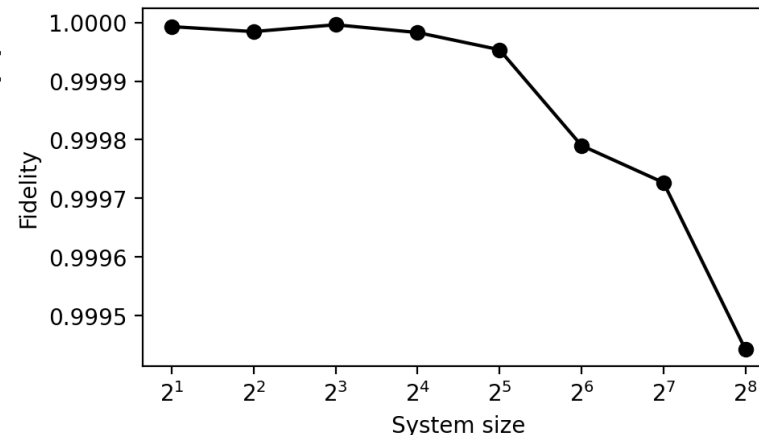
Assessing & generalizing VQLS for fluid dynamics problems

Assessing Variational Quantum Linear Solver for solving canonical fluid flow problems

- Variational Solve $A\mathbf{x} = \mathbf{b}$ as an optimization problem
- Current SOTA scaling on simple linear system (Ising inspired model)
 - $2^{50} \times 2^{50}$ matrix on simulator
 - $2^{10} \times 2^{10}$ matrix on QPU (Rigetti)
- Our implementation (on simulators)
 - $2^{10} \times 2^{10}$ tridiagonal Toeplitz system
 - $2^3 \times 2^3$ canonical fluid flow problem
- **Challenges:**
 - Large # circuits resulting from LC decomposition of A
 - Ansatz selection



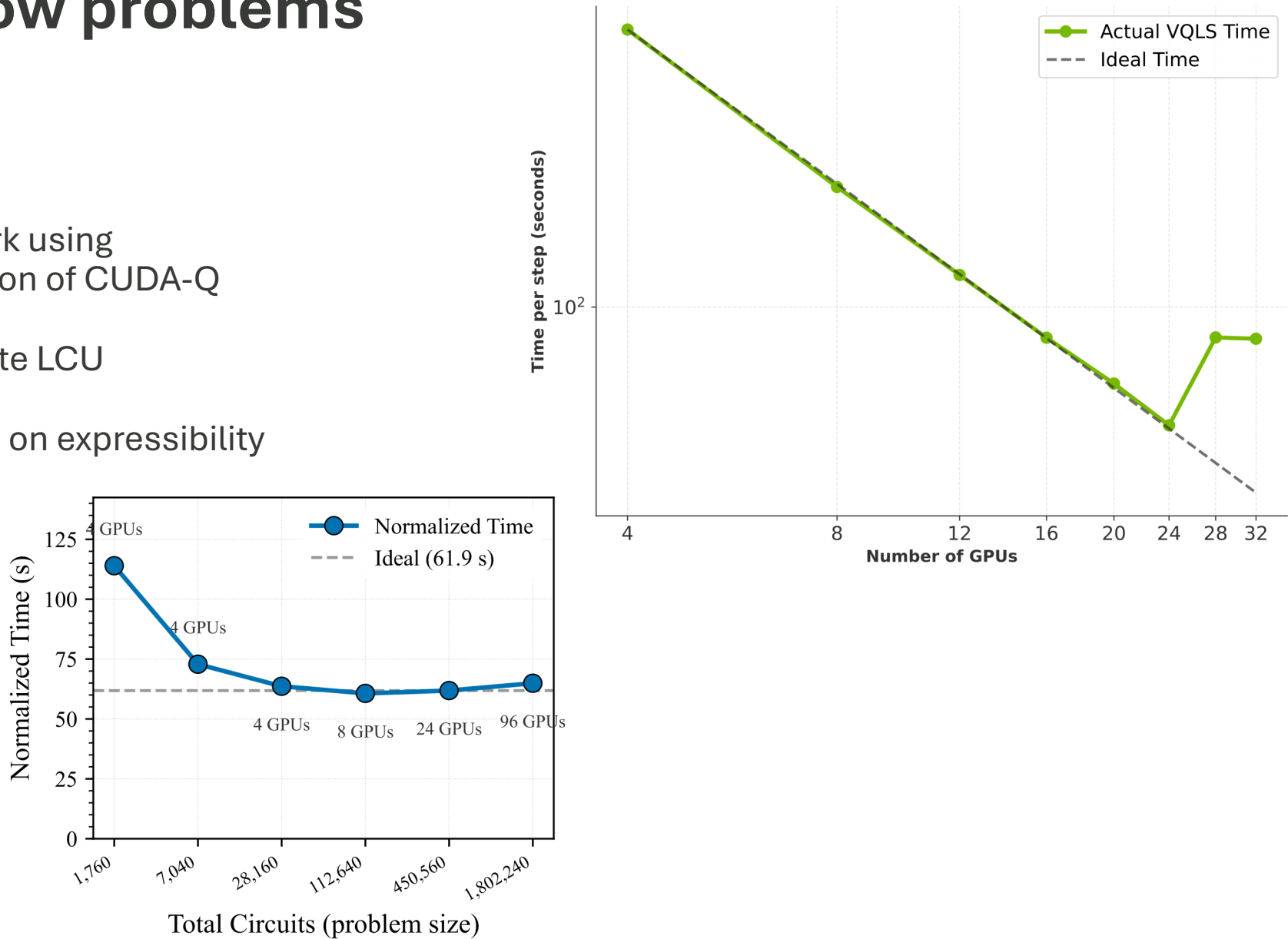
Bravo-Prieto et al., 2023



[Shehata et al., FGCS 2025](#)
[Gopalakrishnan Meena et al., IEEE QCE 2025](#)

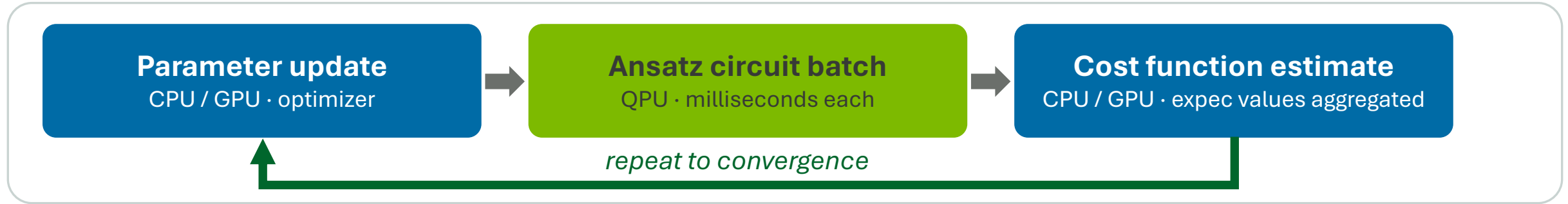
Assessing Variational Quantum Linear Solver for solving canonical fluid flow problems

- Distributed-VQLS – framework using asynchronous circuit execution of CUDA-Q
- Circuit pruning by approximate LCU
- Ansatz characteristics based on expressibility & entangling capability



[Lu et al., accepted in SC26, 2026](#)
Lu et al., accepted in IEEE QCE, 2026

Pattern 2: A tightly coupled loop — the QPU sits inside the optimizer



Characteristics of this mode

- **Queue latency:** every iteration crosses the classical-quantum boundary
- **Per-circuit short execution overshadowed** by
 - # of circuits to be executed
 - convergence not guaranteed
- **D-VQLS:** asynchronous circuit execution
- **Circuit pruning:** approximate LCU representation
- **Gen-VQLS:** training large classical ML models

What the stack must provide

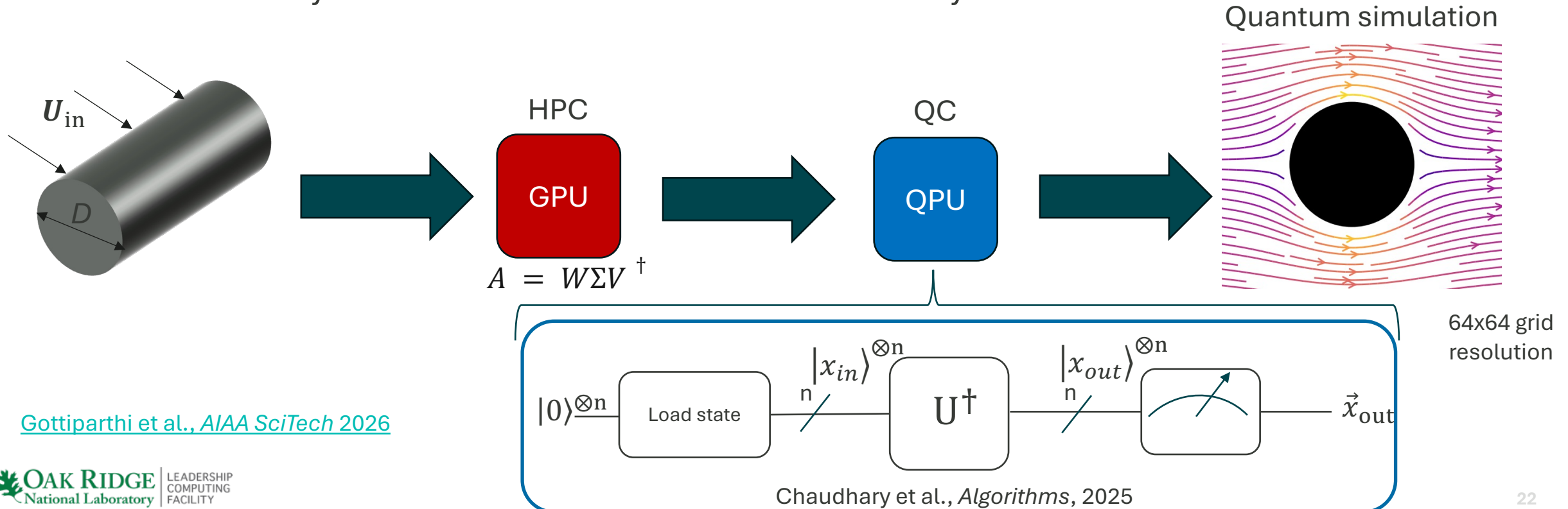
- CPU, GPU & QPU co-scheduled inside one allocation
- Sub-second round trip: an interactive path, not batch submission
- Model-parallel GPUs alongside the QPU for the GenAI variant

Hybrid Quantum Algorithm using Singular Value Decomposition

Solving potential flow past solid bodies using QLSA

SVD-based hybrid quantum algorithm to scale larger problems

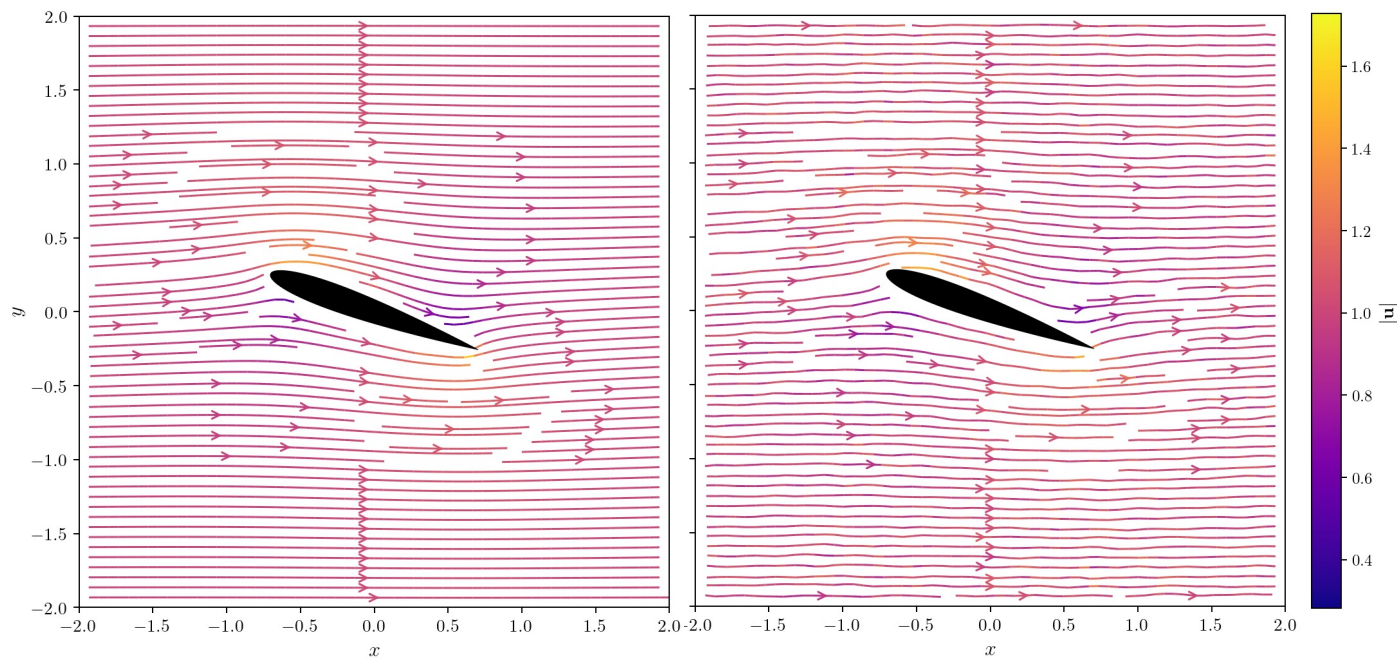
- *Challenges with HHL & VQLS*: Convert inputs to system of choice for the algorithm; Convert to unitary
- Preprocessing system A matrix using SVD for better encoding
 - $Ax = b \rightarrow P U x = b$, where $P^{-1} = W \Sigma^{-1} W^\dagger$ and $U^\dagger = V W$
- Can tackle arbitrary A matrix – need not be Hermitian or Unitary



SVD-based hybrid quantum algorithm to scale larger problems

- Enables assessing hardware & software for larger scale, practically relevant problems
- **Challenge:** Circuit decomposition of arbitrary unitary

Flow over NACA0012 at $\alpha = 20^\circ$, 64×64 grid resolution



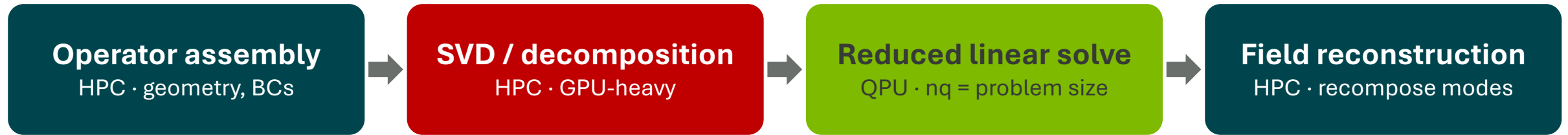
Classical

Quantum



Oak Leaf flow -
Quantum simulation

Pattern 3: A hybrid mode — classical decomposition first, quantum solve second



Characteristics of this mode

- **One-directional, serial:** heavy classical preprocessing
- Classical preprocessing
 - **removes qubit doubling** (20 rather than 40-45 qubits)
 - **circuit depth** still high
- **Current use:** assess hardware & software at practically relevant problem sizes
- **Open cost:** circuit decomposition of an arbitrary unitary

What the stack must provide

- High-throughput classical linear algebra adjacent to the QPU
- GPU-accelerated circuit synthesis & transpilation
- Staging of large operators between parallel file system & QPU host
- Amortization: preprocessing has to cost less than the solve it buys

Quantum-inspired Approach for Nonlinear PDEs



Tensor-network approach for solving PDEs

Tensor network-based solver for nonlinear partial differential equations

$$\nabla \cdot \boxed{u} = 0,$$

$$\boxed{\frac{\partial u}{\partial t}} = - \underbrace{u \cdot \nabla u}_{\text{nonlinear term}} - \underbrace{\nabla p + \frac{1}{Re} \nabla^2 u + s}_{\text{linear terms}}$$

$$\nabla^2 \boxed{p} = - \underbrace{\nabla \cdot (u \cdot \nabla u)}_{\text{nonlinear term}}$$

Quantum **state embedding**

- using **Matrix Product States**

Quantum mapping of **linear** & **nonlinear differential operators**

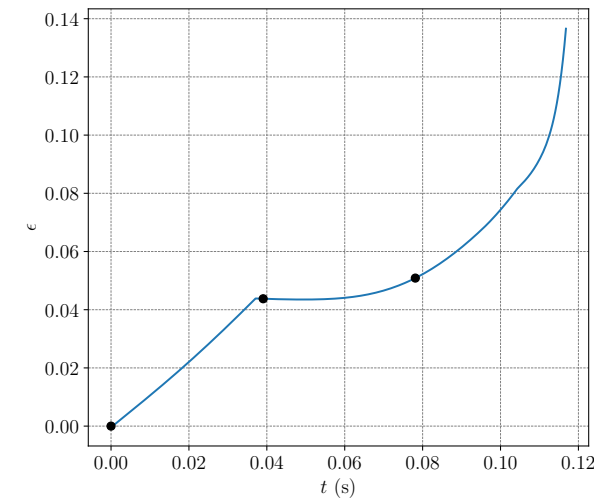
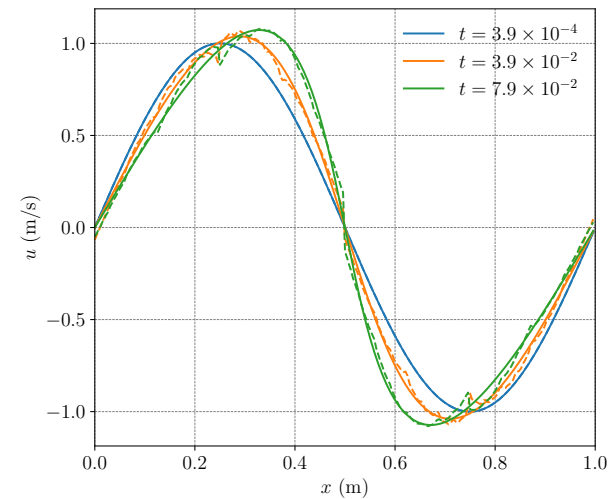
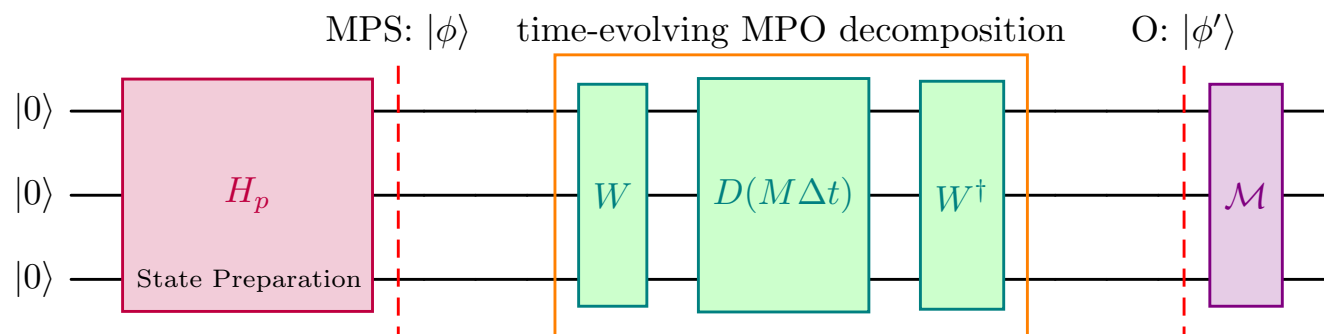
- using **Matrix Product Operators**

LCU for QNPU

Q-Krylov subspace

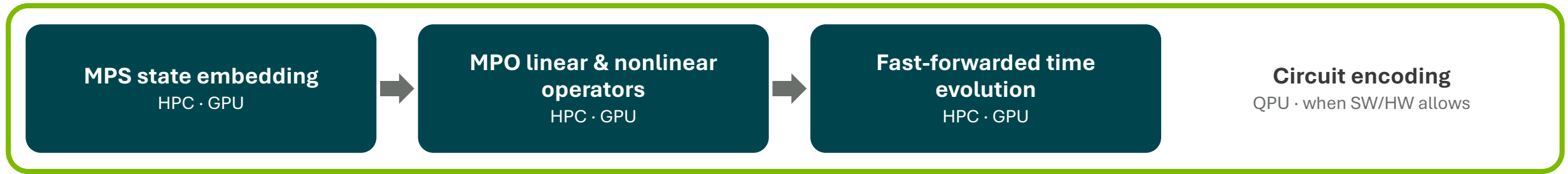
Time evolution of operators

- **Fast Forwarding algorithms**



Gopalakrishnan Meena et al., *IEEE QCE*, 2024
 Gopalakrishnan Meena et al., *AIAA SciTech*, 2026
 Horner et al., *preprint*, 2026

Pattern 4: Quantum-inspired — the tensor-network solver currently runs entirely on HPC



Characteristics of this mode

- **MPS/MPO compression**: by itself source of classical speedup
- **Quantum hardware** is a future backend
- Handles the **nonlinear terms** not addressed by every linear-solver path
- Potential **node-local** relevance:
 - MPS → state-preparation circuits
 - MPO → operator circuits

What the stack must provide

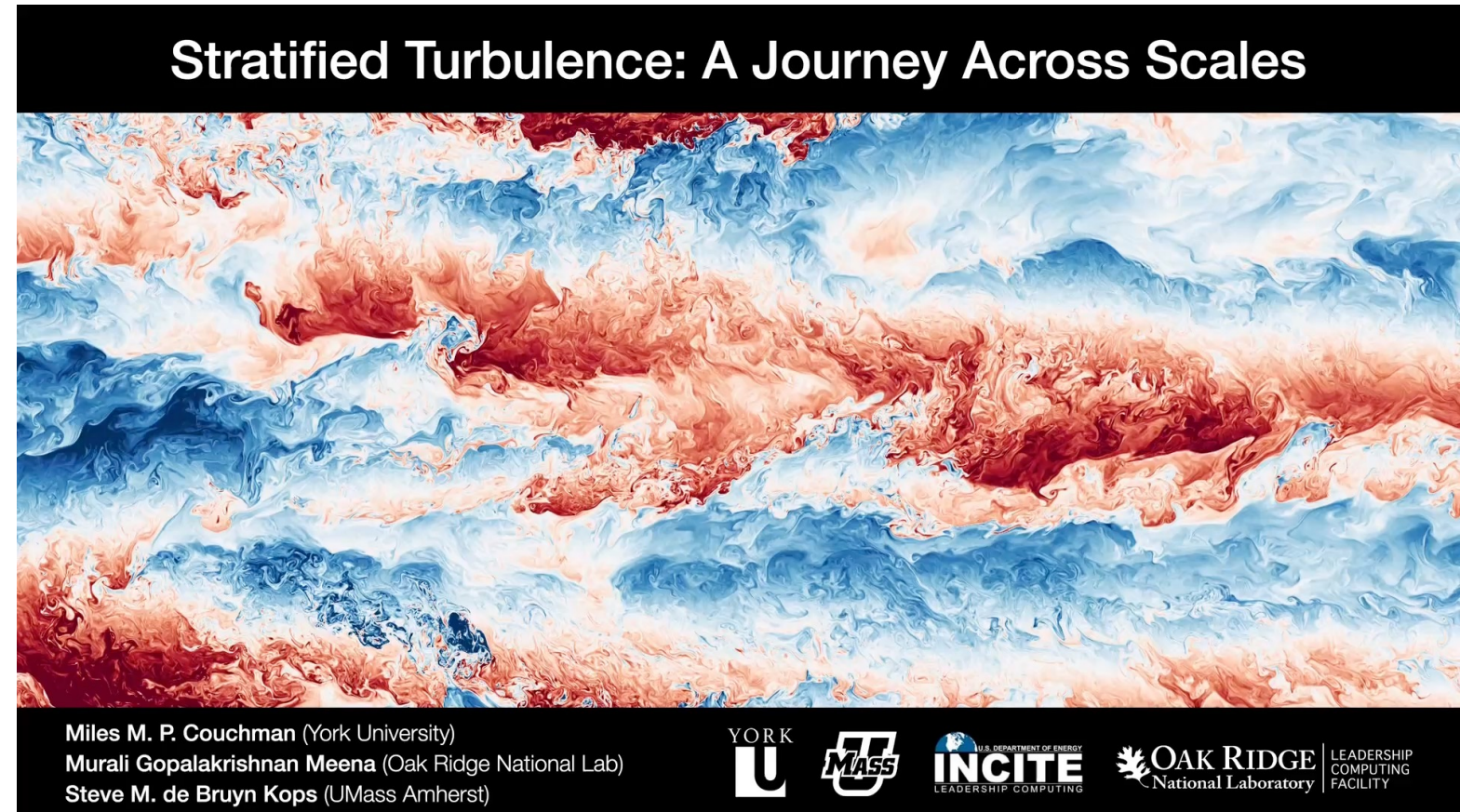
- Scales on GPUs now: benchmarks the encoding before the hardware exists
- Scalable circuit synthesis techniques
- Similar requirements as Approach 1: QPU-only

Scaling to Practical Fluid Dynamics

Beyond 10^{12} grid points: what HPC-QC integration may have to look like

Grid resolution can get prohibitively expensive to simulate for practical fluid flow problems

- 3D stratified turbulence model for oceanographic flow^[1]
- $37k \times 37k \times 4k \approx 6.6 \times 10^{12} > 2^{42}$ grid points
- 100 TB per snapshot
- Turbulence related projects: 35-45% of 2023 OLCF Frontier allocation



At 10^{13} grid points, no path stays standalone — state preparation becomes an HPC job

$$2^{42} - 2^{45}$$

unknowns in the largest differential-equation runs on Frontier

$$2^{40} \times 2^{40}$$

operator that no single node can hold in memory

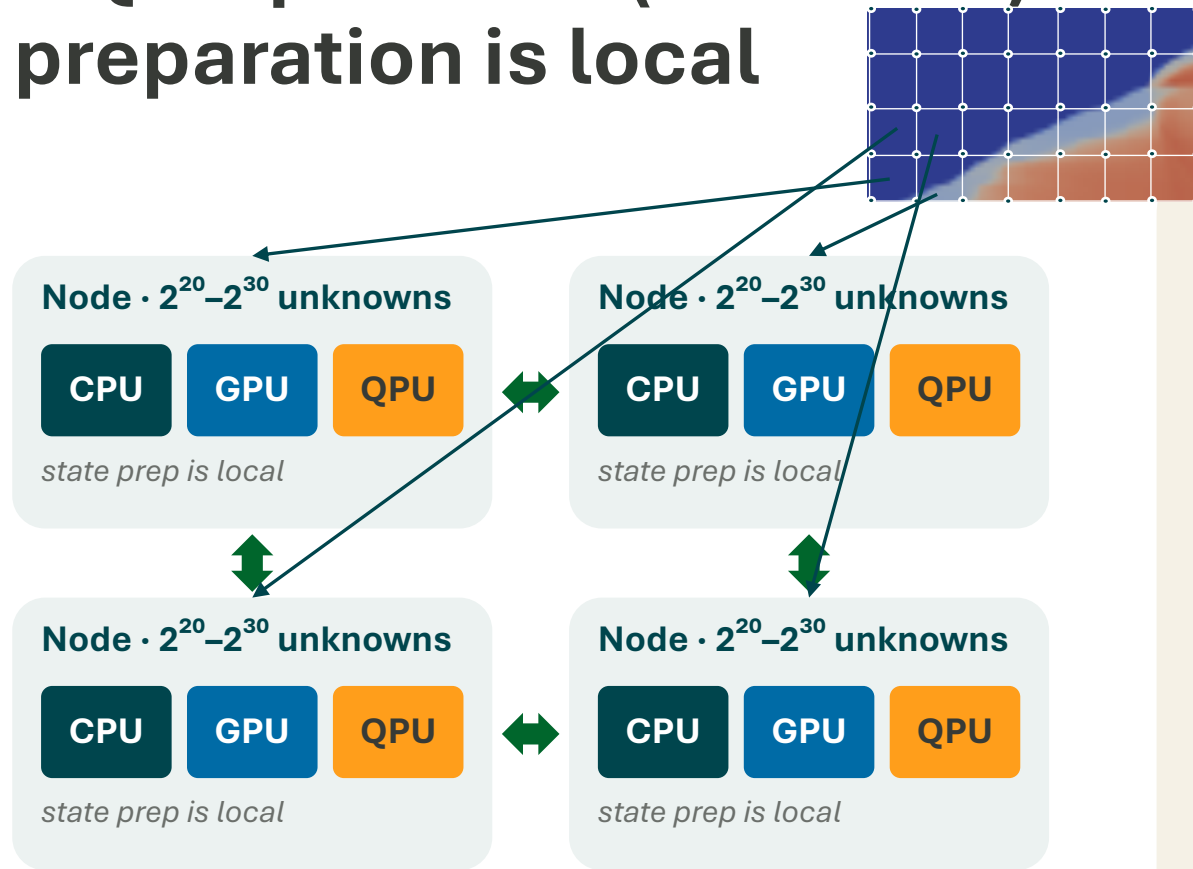
$$10^5 - 10^6$$

gate operations for a single 2^{10} subproblem

Why the previous approaches break

- **Operator is never assembled:** matrix elements must be generated on the fly - a distributed computation
 - Amplitude encoding a 10^{13} grid field is therefore a parallel job in its own
 - State-prep of the “QC-only” pattern acquires a full HPC dependency
- **Problem size not the right parameter** for calling a QPU
 - Classical solvers still win at 2^{20}
 - State preparation caps QC approach
- **Consequence:** a single monolithic QPU attached to a supercomputer might not work for this application

1 QPU per node (as a start): decompose the domain until state preparation is local



green: classical halo exchange between neighbors

Deployment picture

- **Domain-decompose** until each node holds 2^{20} – 2^{30} unknowns: a size where state preparation is a local operation
- Put the **QPU in the node beside the GPUs**: structurally the same move as the one-GPU-per-node machines of the Titan era
- **HPC does what it is good at**: halo exchange, global reductions, load balance
- **Communication pattern** decides feasibility:
 - nearest-neighbor stencils - feasible
 - all-to-all FFT transposes – much harder case

Open question for this consortium: what does a node-local QPU need to expose (API, memory model, scheduling, other?) for a PDE solver to drive it like an accelerator rather than a service?

Dispatching a kernel to QPU: criterion based on complexity class of the kernel, not the size of the problem

Routing inside a running simulation

- **Criterion:** polynomial vs super-polynomial separation for a specific kernel
- **Runtime orchestrator** (CPU or FPGA) routes each kernel per *timestep*: two equations in the same program can route differently based on criterion
- **GPU & QPU run concurrently:** coupled iteratively on approximate solutions rather than blocking on one another
- **Mid-circuit measurement:** would let a time-evolution circuit continue across timesteps instead of restarting

Mode A: submit-and-forget batch

- For problems where the classical run is a month (or years) and the quantum run is hours to days
- Queue latency is irrelevant: throughput & reliability are the key

Mode B: coupled within a run

- Variational (& quantum-ML surrogate) kernels inside a live simulation
- Needs co-scheduling and sub-second round trip: queue latency breaks the application

One facility has to serve both modes: they are not the same machine from the scheduler's point of view.

What we do not know yet (and where we would like help)

Convergence at scale

Does **variational convergence** survive the move from 20 to 40 qubits? Circuit compression is a GPU problem we know how to solve; convergence behavior is not.

State preparation

Distributed, on-the-fly amplitude encoding at 10^{13} points is unsolved and sits in front of every algorithm in this talk.

Node-local QPU semantics

What API, memory model and scheduling contract does a QPU need to expose to be programmed as an in-node accelerator?

Algorithms & Framework, not just Hardware

For this application, the near-term bottleneck is algorithmic & implementation – not just accurate quantum computers.

Community efforts



QC4PDE Consortium
<https://qc4pde.github.io/>

2025

[USNCCM18](#) @ Chicago, Illinois

2026

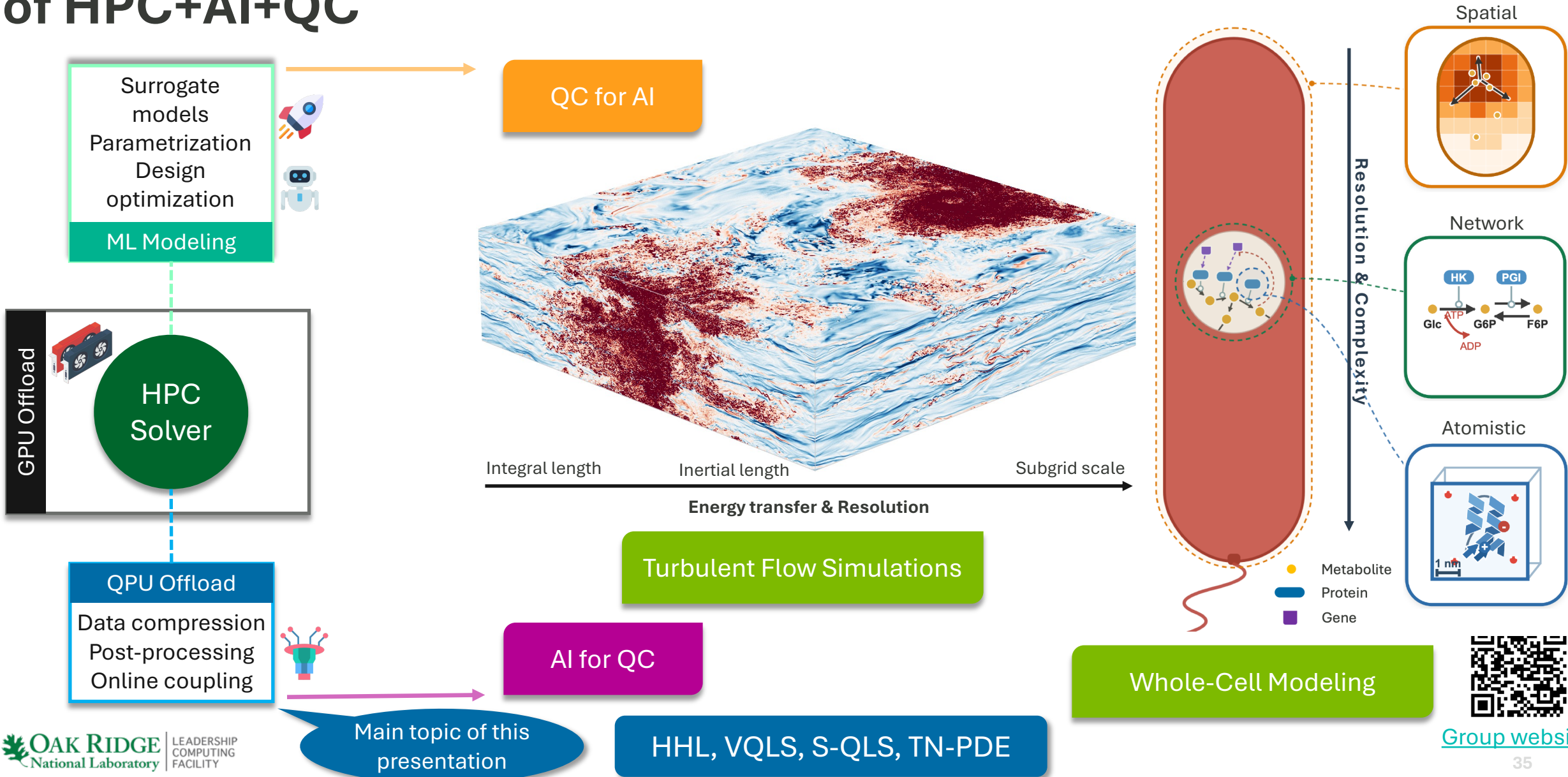
[USNCTAM](#) @ Pasadena, California [June]

[WCCM-ECCOMAS](#) @ Munich, Germany [July]

[QC2026 Workshop](#) @ Nashville, Tennessee [Aug]

[QC4PDE-IEEE QCE 2026](#) @ Toronto, Canada [Sep 13]

Accelerating fluid flow simulations: An integrated ecosystem of HPC+AI+QC



Thank you

